

NEXUS-R Master Specification v8.0

Unified Synthesis of Implementation Plan + v6/v7 Assessment & Redesign

Date: 2026-05-18

Sources: implementation_plan.md + NEXUS-R_v6_Assessment_and_Redesign.pdf

Synthesis Principle: Take the strongest, most specific elements from both documents; discard overlap, marketing fluff, and unproven claims.

1. Core Thesis (What We Are Building)

A personal agent runtime that makes AI task execution progressively cheaper, more private, and more reliable through intelligent routing and verified workflow reuse.

Not “sovereign cognition.” Not a “platform.” A runtime that technical professionals (developers, data scientists, analysts) use 50-200 times per day to execute tasks across terminal, filesystem, browser, and APIs — with full cost control, local-first privacy, and learned efficiency.

The Moat: Every successful execution makes the next one faster and cheaper. Not because of a subscription discount — because the system stops calling expensive APIs for tasks it has already solved.

2. What We Learned From Both Documents

From implementation_plan.md (Strengths):

- **6-module architecture** is the right complexity level for shipping (Intent Parser → ISE → Execution Engine → CPC → Memory Store → Trust Layer)
- **CPC as “cache of proven execution plans”** is the strongest novelty claim
- **Phase-based implementation** (6-week CLI proof-of-concept) is the right approach
- **User-facing cost dashboard** is a genuine UX innovation
- **MCP everywhere + LiteLLM** are correct integration decisions
- **Three-tier memory** (volatile/event/identity) is practical

From v6/v7 Assessment (Strengths):

- **Brutal self-assessment** (5.0/10) shows intellectual honesty
- **Research compilation** (MoMA, OpenHands V1, Claude Code, FlowMind, Shepherd) grounds claims in reality
- **13 specific CONs** with technical reasoning prevent repeated mistakes
- **v7 redesign** (5 subsystems: Input Gateway → CAR → Execution Sandbox → State Core → Workflow Engine) is cleaner
- **Event-sourced single source of truth** replaces overlapping memory layers
- **Deny-first security with ML risk classification** is production-oriented
- **ETD with explicit generalization verification** addresses CPC’s “vaporware” criticism

What Both Got Wrong:

- **Over-optimistic timelines** (6 weeks for Phase 1 is unrealistic; 8-10 weeks is honest)

- **Self-rating inflation** (v7's 9.2/10 is premature; 7.5-8/10 is fair for a design)
- **Business model remains thin** in both documents
- **Some citations may be fabricated** (v6 assessment cites patents/papers that don't verify)

3. The Unified Architecture: 5 Subsystems, 6-Module Detail

We adopt the v7 **subsystem naming** (clearer) but retain the implementation_plan's **module granularity** (more actionable).

NEXUS-R Runtime v8.0
SUBSYSTEM 1: INPUT GATEWAY (Intent Parser)
└ Intent classification (code/research/automation/etc)
└ Confidence estimation
└ Parameter extraction
└ Complexity scoring (1-4 tier)
SUBSYSTEM 2: COGNITION ROUTER (ISE + CAR)
└ Capability profiling (real-time model performance)
└ Multi-objective Pareto routing (cost × latency × privacy)
└ CPC short-circuit (check cache before routing)
└ Adaptive fallback chain with cost caps
└ Result quality tracking for continuous improvement
SUBSYSTEM 3: EXECUTION SANDBOX (Execution Engine)
└ Sandboxed agent loop via MCP (terminal/browser/files)
└ Inline verification at each step
└ ML-based risk classification (local, <10ms)
└ Permission tiers 1-4 with deny-first defaults
└ Event-sourced execution trace
SUBSYSTEM 4: STATE CORE (Memory Store)
└ Event log (append-only, immutable)
└ Three-tier memory: volatile / durable / identity
└ Causal event chaining (parent_event_id)
└ Vector embeddings for semantic retrieval
└ Projected views (UI, billing, audit, debug)
SUBSYSTEM 5: WORKFLOW ENGINE (CPC → ETD)
└ Trace recording from State Core
└ Distillation (extract minimal causal chain)
└ Generalization verification (>90% threshold)
└ Parameterization + deduplication
└ Conservative application (>0.85 similarity, opt-in)
└ Invalidation triggers (failure rate, version changes)
CROSS-CUTTING: TRUST & CONTROL LAYER
└ Permission enforcement


```
"generalization_success_rate": 0.94,  
"last_validated": "2026-05-17T10:00:00Z",  
"avg_cost": 0.0,  
"avg_latency_ms": 4200,  
"invalidation_triggers": ["vercel-cli major version change"],  
"derived_from_events": ["evt_001", "evt_002", "evt_003"]  
}
```

How It Works (Synthesized Pipeline)

1. Trace Recording

- Execution Sandbox records every step to State Core as immutable events
- Causal chaining via `parent_event_id` enables precise replay

2. Distillation (From v7 ETD)

- Extract minimal causal chain (drop dead-ends, retries, exploratory steps)
- Identify which parameters were task-specific vs. task-agnostic
- Mark decision points that had alternatives

3. Generalization Verification (v7's key improvement)

- Test on held-out variants of same task class
- Measure success rate across variations
- **Only admit workflows with >90% generalization success**
- This addresses FlowMind's admitted limitation: "some fine-grained behaviors may be lost"

4. Parameterization + Deduplication (From `implementation_plan`)

- Replace concrete values with typed slots (`/home/user/project` → `{project_dir}`)
- If new ETD matches existing intent signature with >80% step overlap, merge

5. Retrieval (From `implementation_plan`)

- Intent matching: cosine similarity on embeddings
- Context gating: check invariant checks against current environment
- Ranking: `success_rate * recency_weight`
- Threshold: Only ETDs with >85% similarity and >90% success rate eligible

6. Conservative Application (v7's safeguard)

- Opt-in: User must enable workflow suggestions
- Environment match required: same tools available
- Fallback: If any step fails, abandon ETD and fall back to fresh reasoning
- Post-execution: Validate output schema. Update success/failure counts.

7. Invalidation (From `implementation_plan`)

- `failure_rate > 0.3` → mark degraded

- Invalidation triggers fire (dependency version change detected)
- User manual invalidation
- 180 days unused → archive

Why This Is Defensible (Research Positioning)

Prior Art	What They Do	What NEXUS-R ETD Adds
FlowMind (2026)	Distill traces to workflows	Explicit generalization verification with quantified bounds
Shepherd (2026)	Per-step snapshots, trajectory compression	Conservative application thresholds + causal chaining
OpenHands Condenser	2× cost reduction via summarization	Reusable execution plans, not just compressed history
Deep Memory	71% token reduction via bio-inspired compression	Workflow reuse, not just context compression

Research Claim: *Execution trace distillation with verified generalization bounds for autonomous workflow reuse.* The novelty is not “compressing traces” — it’s **admitting only provably generalizable plans into a conservative reuse system.**

5. ISE → CAR: Capability-Aware Routing (Synthesized)

The Problem Both Documents Identified

- MoMA shows routing accuracy depends on task classification granularity
- “Sufficiency” is undecidable a priori — you only know after trying
- OpenHands RouterLLM requires custom `select_llm()` per use case
- Cost of failed weak-model attempts may exceed savings

The Solution: CAR with CPC Short-Circuit + Parallel Probes

Step 1: CPC Check (implementation_plan insight)

- Before any routing, check ETD cache
- If high-confidence match exists → execute directly (often \$0 cost)
- This is the primary cost-saving mechanism

Step 2: Task Classification (From implementation_plan’s 3 dimensions)

Dimension	Measurement	Weight
Reasoning depth	Logical steps, ambiguity, planning need	40%
Knowledge breadth	Domain specificity, recency	30%
Tool complexity	Number/type of tools, irreversibility	30%

- Small, fast classifier (rule-based initially, fine-tuned small model later)
- Scores 1-4 per dimension → composite task tier (T1-T4)

Step 3: Capability Profiling (v7's key addition)

```
provider_profile = {
  "model_id": "claude-sonnet-4.1",
  "capabilities": {"code": 0.92, "math": 0.88, "writing": 0.95, "reasoning": 0.90},
  "latency_p50_ms": 1200,
  "latency_p99_ms": 4500,
  "cost_per_1k_tokens": 0.003,
  "privacy_level": "cloud", # local | private_server | cloud
  "tool_format": "anthropic",
  "context_window": 200000
}
```

- Profiles updated based on observed outcomes (feedback loop)

Step 4: Multi-Objective Routing (v7's Pareto optimization) Optimize across:

- Capability match (estimated vs. observed)
- Latency requirement (user-specified or inferred)
- Cost constraint (per-request budget or user tier)
- Privacy requirement (hard constraint, not preference)
- Verification confidence (high-certainty tasks → stronger models)

Step 5: Adaptive Fallback Chain (Synthesized)

```
local_7b ($0, max privacy) → local_14b ($0, max privacy) → local_70b ($0, max privacy)
  → BYOK_budget ($user_cost, user control) → BYOK_frontier ($user_cost, user control)
  → managed_premium ($platform_cost, standard privacy)
```

- Cost caps at each tier
- Never jump more than one tier at a time
- User gate at managed_premium (always requires approval)

Step 6: Parallel Probe (implementation_plan's uncertainty handling)

- When uncertain, send to T1 and T2 simultaneously
- If T1 succeeds verification → discard T2 result
- Cost is low (T1 is local/free)

Step 7: De-escalation Learning

- If task consistently succeeds at T1 that was classified T2 → update classifier
- Over time, more tasks route locally as ETD accumulates

Performance Targets (Honest, From v7)

Metric	Target	Rationale
Routing overhead	<50ms	Research shows routing is cheap vs. LLM inference (2-15s)
End-to-end latency (simple)	<3s	Flat architecture, no hierarchical coordination
Cost reduction vs. always-frontier	60-80%	MoMA and ICA AI demonstrate this range
Local mode capability	80% of cloud on common tasks	70B models + ETD reuse for standard patterns

6. Security & Trust (Synthesized from Both)

Permission Model: Capability-Based, 4 Tiers

Tier	Actions	Approval
T1	Read files, read-only terminal (ls, cat, grep), web search, safe MCP tools	Auto
T2	Write/modify workspace files, sandboxed terminal, HTTP GET to approved domains	Auto + log
T3	Arbitrary terminal, files outside workspace, HTTP POST/PUT/DELETE, browser actions, API keys	User prompt
T4	Delete files, access secrets, deploy to prod, financial transactions, irreversible actions	Explicit confirm + reason

Sandboxing (From v7 + implementation_plan)

- **Terminal:** Docker container with mounted workspace only; fallback to OS-level restrictions
- **Browser:** Isolated Chromium, no host filesystem access, no real browser profile
- **Filesystem:** Scoped to workspace; parent traversal blocked
- **API calls:** Whitelisted domains only; credential injection via SecretRegistry

ML Risk Classification (v7's innovation)

```
risk_score = f(action_type, target_resource, permission_scope, user_history)
# Low risk → auto-execute
# Medium risk → prompt user
# High risk → require explicit override + audit log
```

- Runs locally (no cloud dependency for security decisions)
- <10ms per action

Audit & Rollback (From implementation_plan)

- Every action: {timestamp, action_type, tool, input_summary, output_summary, model_used, tier, cost, user_approved, verification_result, action_hash}
- File operations: snapshot before write/delete (git stash or copy)
- Terminal: record stdout/stderr; provide undo suggestions
- Retention: 90 days default, configurable

Secret Handling

- API keys in OS credential manager (Windows Credential Store / macOS Keychain / Linux libsecret)
- Never in logs, audit trails, or ETD records
- Never sent to models; injected at execution layer only
- BYOK keys validated on entry (test API call) + rotatable

Prompt Injection Defense (implementation_plan)

1. **Input framing:** External content wrapped in <EXTERNAL_DATA> delimiters
2. **Output validation:** Parse structurally (JSON tool calls), never eval()
3. **Action allowlisting:** Only declared tools callable; no dynamic creation
4. **Anomaly detection:** TIER 4 requests during TIER 1 task → flag and pause

7. State Core: Three-Tier Memory (Synthesized)

Tier	Name	What It Stores	Lifecycle	Storage
Volatile	Working State	Current task context, active plan, tool results, conversation	Dies when task completes	In-memory
Durable	Event Store	Immutable log of all actions, decisions, outcomes, ETD derivations	Append-only, 90 days default	Local SQLite + optional sync
Identity	User Profile	Preferences, tool configs, API key refs, behavioral policies, classifier weights	Slow-changing, user-editable	Local encrypted file

Key Design (From v7)

- **Single source of truth:** Event Store is the only mutable state
- **Causal chaining:** parent_event_id links events into provenance chains
- **Vector embeddings:** Every event has embedding for semantic retrieval
- **Projected views:** Read-optimized views for UI, billing, audit, debugging (derived from event log)
- **Compaction:** Deep Memory-style semantic compression (71% token reduction demonstrated)

What Was Removed

- **✗ World Model** (too ambitious, replaced by scoped task state in Working State)
- **✗ Personal Cognition Layer** (merged into Identity tier + ETD)
- **✗ Separate Persistent Workflow Memory** (ETD is derived from Event Store, not a separate layer)

8. Implementation Roadmap: Honest Phases

Phase 1: Core Runtime (Weeks 1-8) — SHIP THIS

Goal: Can a user give a natural language task and have it executed locally or via BYOK, with full audit trail?

- Intent Parser (basic: regex + keyword classification, 4-tier scoring)
- Execution Engine with MCP (terminal + filesystem tools only)
- LiteLLM integration (local via Ollama + one BYOK provider)
- Basic CAR: 2 tiers only (local vs. BYOK), no CPC yet
- Event-sourced execution log (SQLite)
- Trust: T1-T2 permissions, workspace sandboxing (Docker or chroot)
- CLI interface (minimal, functional)
- Cost tracking (per-request, no dashboard yet)

Team: 2-3 engineers

Validation: 5 beta users run 20+ tasks each; measure success rate and cost

Phase 2: ETD + Full CAR (Weeks 9-16) — PROVE THE VALUE

Goal: Does ETD measurably reduce cost and latency on repeated tasks? Do users feel in control?

- ETD extraction from execution logs
- ETD retrieval and replay with conservative thresholds
- Full CAR: 4 tiers (local/BYOK-budget/BYOK-frontier/managed)
- Adaptive fallback chain with cost caps
- Cost dashboard UI (per-task, cumulative, model used, tier)
- Trust: T3-T4 permissions, ML risk classifier (local, rule-based)
- Web UI (basic, read-only audit log + cost view)
- Parallel probe for uncertain tasks

Validation: A/B test: 10 users with ETD enabled vs. 10 without; measure cost reduction and latency

Phase 3: Browser + Polish (Weeks 17-24)

- Browser automation via MCP (Playwright)
- Three-tier memory system (volatile + durable + identity)
- ETD composition (chain multiple ETDs)
- User preferences and personalization (Identity tier)
- Execution replay and fork from event log
- Managed premium tier integration
- Polished Web UI

Phase 4: Community + Scale (Weeks 25+)

- ETD community exchange (opt-in sharing, privacy-stripped)
- Multi-provider premium tier
- Advanced CAR with learned classifier
- API for third-party integrations
- Mobile companion (view audit, approve escalations)

9. Economic Model (What Both Documents Underdeveloped)

Revenue Streams

Stream	Description	Target User
Managed Premium	Platform-negotiated rates, reserved capacity, SLA	Teams/enterprises without BYOK
ETD Community	Marketplace for verified workflows (revenue share)	Power users
Enterprise License	On-prem deployment, audit compliance, SSO	Regulated industries

Cost Structure

Cost	Mitigation
Free local mode support burden	Self-service docs, community Discord; no 1:1 support
BYOK users generate no direct revenue	Upsell to managed premium; ETD marketplace fees
Premium tier competes with BYOK	SLA guarantees, reserved capacity, priority support

Unit Economics (Hypothesis — Requires Validation)

- CAC: Low (developer tool, organic growth via GitHub/open source)
- LTV: High (sticky once ETD accumulates; switching means losing learned workflows)
- Gross margin on managed premium: 30-40% (platform-negotiated rates vs. retail API pricing)

10. Competitive Positioning

Competitor	Their Strength	Our Differentiation
Claude Code	Deep reasoning, terminal-first	Local-first + BYOK + cost control + workflow reuse
OpenHands	Research-grade autonomy	Production security + user-facing cost routing + ETD
Cursor/Windsurf	IDE-native, great UX	General-purpose execution (not just coding) + local models
Jan.ai/LM Studio	Local inference UI	Agent execution + tool use + workflow memory
n8n/Zapier	Visual workflow automation	LLM reasoning + adaptive routing + local execution
MoMA (research)	Pareto-optimal routing	User-facing control + privacy hard constraints + ETD integration

The Pitch: “The AI agent that gets cheaper the more you use it.”

11. Risk Register (Honest, From Both Documents)

Risk	Severity	Likelihood	Mitigation
ETD doesn't generalize (workflows too variable)	High	Medium	Start narrow (deploy, test, data pipeline). Measure empirically. Kill if <70% success.
Local models too weak for useful execution	Medium	Medium	Qwen 2.5 7B/Phi-4 already competent for simple tasks. CAR handles the gap.
Big players build this (OpenAI, Anthropic)	High	High	Ship fast. ETD community creates network effects they can't replicate.
Users don't trust local agents with file/terminal	Medium	Medium	Strong permissions + audit + rollback. Start read-only.
Scope creep kills focus	High	High	Phase 1 = CLI + terminal + files ONLY. No browser, no APIs, no premium.
Routing overhead adds latency	Low	Low	Target <50ms. Research confirms routing is cheap vs. inference.
Security breach in sandbox	High	Low	Docker isolation + deny-first + ML risk classifier. Bug bounty program.
BYOK key compromise	Medium	Low	OS credential manager + per-session isolation + audit logging.

12. Patent Strategy (Consolidated)

Primary Claim: ETD with Verified Generalization

- Execution trace distillation with explicit generalization testing
- Conservative application with similarity + success rate thresholds
- Causal event chaining for provenance
- **File as utility patent**

Secondary Claim: CAR with Privacy-Hard-Constraint

- Multi-objective Pareto optimization with privacy as non-negotiable constraint
- Adaptive capability profile updating based on observed outcomes
- Cost-capped fallback chains with user-controlled limits

- CPC short-circuit before routing decision
- **File as continuation**

Supporting Documentation

- MoMA paper (arXiv:2509.07571v1) — establishes routing as novel field
 - OpenHands V1 SDK paper (arXiv:2511.03690v1) — establishes event-sourced state as prior art
 - FlowMind/Shepherd papers — establish trace distillation as active research area with open problems
 - Internal benchmarks: ETD success rates, cost reduction measurements
-

13. Success Metrics (What “Done” Looks Like)

Phase 1 Success

- 5 beta users, 20+ tasks each, >80% success rate
- Tasks execute locally (T1) or BYOK (T2-T3) with full audit trail
- Average cost per task < \$0.10 for T1-T2

Phase 2 Success

- ETD reduces cost by >40% on repeated tasks (measured vs. Phase 1)
- Users can explain why a task cost what it did (cost dashboard)
- Zero unauthorized T4 actions (security test)

Phase 3 Success

- Full end-to-end: terminal + files + browser + APIs
- ETD composition works (chain 2+ workflows)
- 100+ ETDs in community exchange

Business Success (12 months)

- 1,000 active users
 - 50% of tasks routed locally (privacy + cost win)
 - \$10K MRR from managed premium + ETD marketplace
-

14. What We Are NOT Building (Anti-Requirements)

- **✗ Model Council** — Debating among models is too expensive, unproven
 - **✗ World Model** — Complete state representation of “the world” is open research
 - **✗ Formal Verification Layer** — Symbolic verification doesn’t scale; use deny-first + ML risk
 - **✗ Custom Provider Abstraction** — LiteLLM already handles 100+ providers
 - **✗ Separate Search/Retrieval Layer** — Search is a tool via MCP, not a layer
 - **✗ Sovereign Cognition Branding** — Users buy outcomes, not concepts
 - **✗ 12-Layer Architecture** — Complexity is the enemy of shipping
-

15. Final Verdict

NEXUS-R v8.0 is a 7.5/10 design with a 9/10 core idea.

The core idea — **verified workflow reuse making AI execution progressively cheaper** — is genuinely novel and defensible. The architecture is clean enough to ship (5 subsystems, 6 modules). The implementation roadmap is honest about what can be built in 8 weeks vs. 8 months.

The risk is not technical. It's focus. The biggest threat to this project is scope creep — adding “just one more layer” or “just one more feature” until it becomes the 13-layer v6 that never ships.

The mandate: Build Phase 1 in 8 weeks. Measure cost reduction. If ETD shows >40% savings on repeated tasks, double down. If not, kill it. A focused 6-module CLI that demonstrably saves users money is worth more than a 12-layer diagram that never compiles.

“The AI agent that gets cheaper the more you use it.” That's the pitch. That's the moat. Everything else is scaffolding.